ICT1012

OPERATING SYSTEMS

LABORATORY MANUAL

Lab3-w8: Lab Assignments

AY 2025/26

BACHELOR OF ENGINEERING

IN

INFORMATION AND COMMUNICATIONS TECHNOLOGY

Contents

Lab Assignment 3
Prerequisite 3
Submission deadline 3
Assignment Task 1: Extending the xv6 File System (Large Files) 3
Task introduction 3
Implementation 6
Testing the task 8
Assignment Task 2: Symbolic links 9
Task introduction 9
“user/symlinktest”: User space test program to test the implementation of “symlink” 10
Implementation 11
Testing the task 13
Run all test cases 14
Assignment submission 14

Lab Assignment

Prerequisite

- Download "**lab.zip**" from "xSiTe" folder "**Labs/xv6labs-w8: FS**" to "**c:/ICT1012/xv6labs-w8**".
- Unzip and keep all folders and files directly under "**c:/ICT1012/xv6labs-w8**" without the "labs" sub-folder.

Submission deadline

You must submit your result to Gradescope before 13 March 2026 11:59pm

Assignment Task 1: Extending the xv6 File System (Large Files)

Task introduction

This task is to increase the maximum size of an xv6 file. Currently, xv6 limits files to a tiny **268 blocks**, or 268*BSIZE bytes (BSIZE is **1024** in xv6), i.e. **~268 KB**.

In xv6, every file is managed by an "***inode***" (Index Node). Think of the "*inode*" as a fixed-size Catalog Card stored on the disk.

The "*inode*" has a limited number of slots to list where the file's data blocks are located. This limit comes from the fact that an xv6 "*inode*" contains 12 "**direct**" block numbers and one "**singly-indirect**" block number, which refers to a block that holds up to 256 more block numbers, for a total of 12+256=268 blocks.

This task is to "remodel" this Catalog Card ("*inode*") to support **65,803 blocks** (~64 MB) using "**doubly-indirect**" blocks.

1. The Narrative: Before vs. After

Refer to the definition of NDIRECT (direct blocks), NINDIRECT (singly-indirect blocks), MAXFILE and the "*dinode*" data structure in "**kernel/fs.h**" below

```
//kernel/fs.h

#define NDIRECT 12
#define NINDIRECT (BSIZE / sizeof(uint))
#define MAXFILE (NDIRECT + NINDIRECT)

// On-disk inode structure
struct dinode {
  short type;       // File type
  short major;      // Major device number (T_DEVICE only)
  short minor;      // Minor device number (T_DEVICE only)
  short nlink;      // Number of links to inode in file system
  uint size;        // Size of file (bytes)
```

```
 uint addrs[NDIRECT+1];   // Data block addresses
};
```

The "Before" Layout (NDIRECT 12)

- **Slots 1-12 (Direct):** Points directly to 12 data blocks.
- **Slot 13 (Singly-Indirect):** Points to one **"Map Book"** (which holds 256 addresses).
- **Total:** 12 + 256 = 268 blocks.

Make sure "$U/_bigfile\" is added to Makefile to compile.

Before performing any changes, run the test "*bigfile*". This user space program will try to create a big file named "*big.file*" that will need **65,803 blocks.** The OS hits the limit of the Catalog Card (268 blocks) and stops:

```
$ make clean && make qemu
...
nmeta 46 (boot, super, log blocks 30 inode blocks 13, bitmap blocks 1) blocks 1954
total 2000
...
xv6 kernel is booting
init: starting sh
$ bigfile
..
wrote 268 blocks
bigfile: file is too small
$
```

Explanation for the output above:

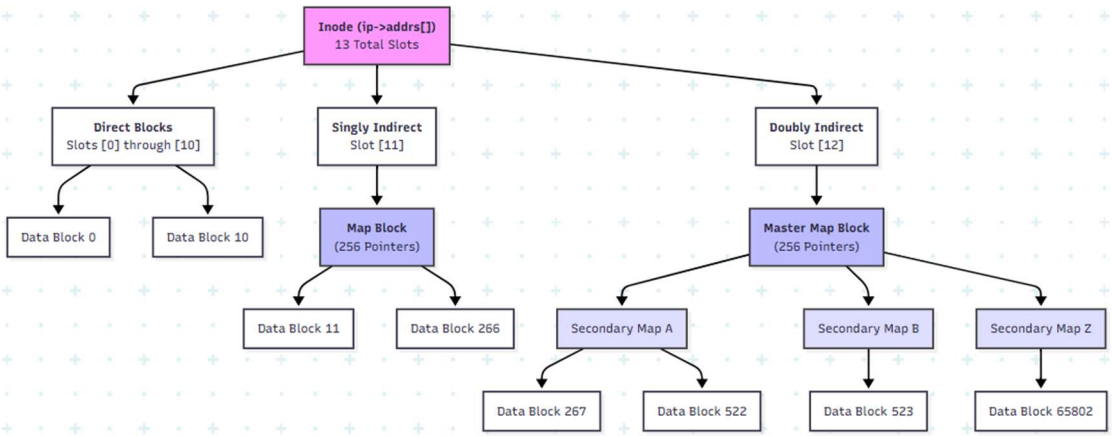
- The "*user/bigfile.c*" program is a "stress test" for the file system. Its primary purpose is to verify that whether the kernel can handle a file significantly larger than the original 268-block limit.
- The program starts by creating a file named "*big.file*" and enters an infinite while loop to write data. In every block, it stores the current block number (blocks) at the very beginning of the buffer. This ensures every block has unique, verifiable data.
- It keeps writing until the "*write()*" system call returns zero or a negative number, meaning the file system is full or the maximum file size has been reached.
- Each dot (.) printed to the console represents 100 blocks of data successfully written to the disk.
- The message "wrote 268 blocks" signifies that the test failed because "*bigfile*" expects to be able to create a file with 65803 blocks, but unmodified xv6 limits files to 268 blocks.

To support larger files, we cannot make the "*inode*" physically larger. Instead, we change how we use the **13 slots** available on the card.

The "After" Layout (NDIRECT 11)

We sacrifice one "Direct" slot to create a "Doubly-Indirect" system.

- **Slots 1-11 (Direct):** Points directly to 11 data blocks.
- **Slot 12 (Singly-Indirect):** Points to one **"Map Book"** (256 addresses).
- **Slot 13 (Doubly-Indirect):** Points to a **"Master Map Book"**.
    - The Master Map Book points to **256 Secondary Map Books**.
    - Each Secondary Map Book points to **256 Data Blocks**.
- **Total:** 11 + 256 + (256 * 256) = 65,803 blocks.



2. The Toolkit: "*bread()*" and "*brelse()*"

When navigating the file system, you cannot read a "Map Book" directly from the disk. You must bring it into the "Reading Room" (the Kernel's Memory).

- ***"bread(device, block_number)"***: This "opens" a map book. It fetches the block from the disk and puts it into a **buffer**. You can then look inside to see the addresses.
- ***"brelse(buffer)"***: This "closes" the map book. The Kernel has a very small "Reading Room" (buffer cache). If you open books without closing them, the Kernel will run out of space and crash (panic). Release a locked buffer. Move to the head of the most-recently-used list. ***"brelse"* must be always called as soon as you have the address you need.**

Other related tools:

balloc(device): Allocates a zeroed disk block.

bfree(device, block): Free a disk block.

Implementation

1. Update "***kernel/param.h***"
   Change **FSSIZE** from 2000 to 200000

   Note: You should see the following output from "*mkfs/mkfs*" in the make output:

   ```
   nmeta 70 (boot, super, log blocks 30 inode blocks 13, bitmap blocks 25) blocks 199930 total 200000
   ```

   This line describes the file system that "*mkfs/mkfs*" built: it has 71 meta-data blocks (blocks used to describe the file system) and 199,929 data blocks, totaling 200,000 blocks.

2. Update "***kernel/file.h***"
   Ensure "***struct inode***" has "***addrs[NDIRECT+2]***"
   Note: "*struct inode*" defines how the "*inode*" looks in the computer's memory (RAM).

3. Update "***kernel/fs.h***"
   Set NDIRECT to 11
   Define NDINDIRECT
   Update MAXFILE
   Ensure struct dinode has addrs[NDIRECT+2]
   Note: "*struct dinode*" defines how the "*inode*" looks on the physical disk

4. Update the Map Reader: "***bmap()***" in ***"kernel/fs.c***"

   The "***bmap()***" function is the OS's navigator. It takes a logical block number (e.g., "I want block 500") and finds the physical disk address.

   Tips: In "*bmap()*", review the code on how the direct and singly-indirect block allocation are done. Doubly-indirect block allocation needs an extra step compared to the singly-indirect block allocation. Add code for doubly-indirect block allocation after the direct and singly-indirect block allocation.

   The Logic for Doubly-Indirect Blocks:

   a) **Check Range:**

      If the block number is beyond the Direct and Singly-Indirect limits, enter the Doubly-Indirect logic.

   b) **Level 1 (Master Map):**
      - Get the address of the doubly-indirect block:

        Look at the "*inode's*" 13th slot. If it's 0, call "*balloc()*" to create the Master Map.

      - Load the doubly-indirect block (the first level):

"*bread()*" the Master Map.

c) **Calculate Indices:**
   - Index1 (calculate index for the first level): Which "Secondary Map" do we need? (Calculated by **bn / 256**; (e.g., "I want block 500") then 500/256 = 1.95 ~= 1)
   - Index2 (calculate index for the second level): Which "Data Block" inside that secondary map? (Calculated by **bn % 256**; (e.g., "I want block 500") then 500%256 = 244)

d) **Level 2 (Secondary Map):**
   - Check Index1 inside the Master Map. If empty, call "***balloc()***" to create a Secondary Map. Call "***log_write()***" to mark the buffer to be written to the log.
   - **"*brelse()*"** the Master Map (always close the previous book!).
   - Load the second-level indirect block: **"*bread()*"** the Secondary Map.

e) **Level 3 (Data Block):**
   - Check Index2 inside the Secondary Map. If empty, call "***balloc()***" for the final data block. Call "***log_write()***" to mark the buffer to be written to the log.
   - **"*brelse()*"** the Secondary Map.

f) **Return the final data block address.**

5. Update the "***Cleaner***": "***itrunc()***" in "***kernel/fs.c***"

   When a file is deleted, "*itrunc()*" must return all used blocks to the "Free Pool" so the disk doesn't fill up.

   The Logic for Cleaning:

   a) **Direct & Singly-Indirect:** (Already handled in original code).
   b) **Doubly-Indirect Tree Walk:**
      - **"*bread()*"** the Master Map Book.
      - **Loop 1:** For each entry in the Master Map:
        - If the entry is not empty, "***bread()***" that Secondary Map Book.
        - **Loop 2:** For each entry in the Secondary Map:
          - If the entry is not empty, call "*bfree()*" on that Data Block.
        - **"*brelse()*"** the Secondary Map, then call "*bfree()*" on the Secondary Map block itself.

- **“*brelse()*”** the Master Map, then call “*bfree()*” on the Master Map block itself.

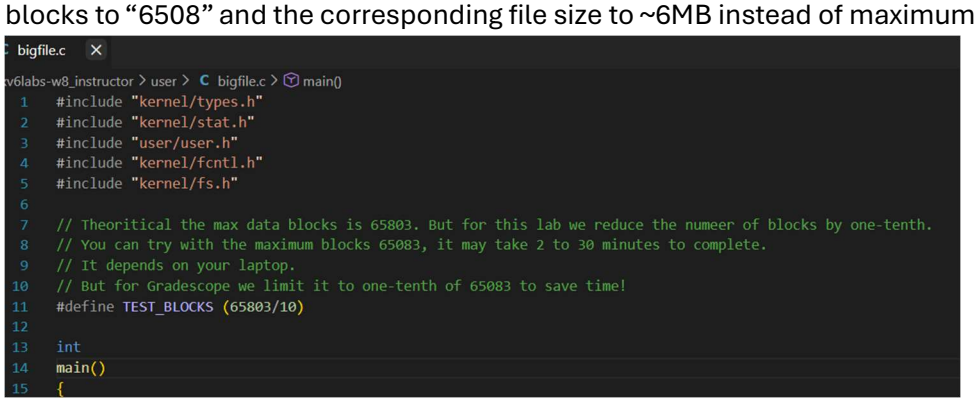
c) **Clear the “*inode*”:** Set the 13th slot in the “*inode*” back to 0.

6. Hints
   - If you change the definition of NDIRECT, you'll probably have to change the declaration of “*addrs[]*” in “*struct inode*” in “*file.h*”. Make sure that “*struct inode*” and “*struct dinode*” have the same number of elements in their “*addrs[]*” arrays.
   - If you change the definition of NDIRECT, make sure to create a new “*fs.img*”, since “*mkfs*” uses NDIRECT to build the file system.
   - If the file system gets into a bad state, perhaps by crashing, delete “*fs.img*” (do this from Unix, not xv6). make will build a new clean file system image.
   - Don't forget to “*brelse()*” each block that you “*bread()*”.
   - You should allocate indirect blocks and doubly-indirect blocks only as needed, like the original “*bmap()*”.
   - Make sure “*itrunc()*” frees all blocks of a file, including double-indirect blocks.

Testing the task

1. Run “**make clean && make qemu**”. Then run “**bigfile**”

   Note: As stress testing with maximum blocks (65803) can take any time between 2 to 35 minutes to complete the test depending on the hardware and software environment, we restrict “*bigfile.c*” to stress to one-tenth of the theoretical maximum, we reduce the max blocks to “6508” and the corresponding file size to ~6MB instead of maximum ~64MB:



   Note: This may take more than 2 minutes.

```
$ make clean && make qemu
xv6 kernel is booting

init: starting sh
$ bigfile
.................................................................
wrote 6580 blocks
bigfile done; ok
$
```

   Success: 65 dots followed by message “wrote 6580 blocks”.

```
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2305
cat            2 3 33312
echo           2 4 32160
forktest       2 5 16032
grep           2 6 36728
init           2 7 32608
kill           2 8 32088
ln             2 9 31896
ls             2 10 35400
mkdir          2 11 32144
rm             2 12 32128
sh             2 13 54720
stressfs       2 14 33008
usertests      2 15 138352
grind          2 16 48504
wc             2 17 34280
zombie         2 18 31488
bigfile        2 19 32680
symlinktest    2 20 44400
bigfile        2 19 32680
console        3 22 0
big.file       2 23 6737920
$
```

2. Run **"./grade-lab-fs bigfile"**

```
$ ./grade-lab-fs bigfile
make: 'kernel/kernel' is up to date.
== Test running bigfile == running bigfile: OK (53.1s)
$
```

Assignment Task 2: Symbolic links

Task introduction

The Task1 (Large Files) is about how to handle data (making files bigger). Task 2 ("Symlinks") is about how to handle metadata by creating Symbolic Links (indirection, making files point to other files).

The Concept: The "Shortcut" Analogy

A **Symbolic Link (*"symlink"*)** is a special type (T_SYMLINK) of file that doesn't contain data itself. Instead, it contains a **string** (a path) to another file. When you try to open it, the kernel sees it is a link, reads the path inside, and opens that target instead.

- **Normal File:** Like a physical box containing a toy.

- **Symlink:** Like an empty box containing a sticky note that says, *"The toy is actually in the cabinet under the sink."*

The "Breadcrumb" Logic

1. **Open "Link B"** → Kernel sees it is a Symlink.
2. **Read Target** → Kernel finds the string "File A" inside.
3. **Redirect** → Kernel automatically switches to opening "File A".
4. **Loop Protection** → Kernel stops after 10 jumps to prevent infinite recursion (A → B → A).

Analogy: A Symlink is a **postal forwarding address**. You send mail to the old house; the post office sees the note and delivers it to the new one.

Core Uses

- **Version Control:** Point python3 → python3.11. Upgrade the version without changing the command.
- **Space Efficiency:** Reference a 10GB file in multiple folders without duplicating the data.
- **Stability:** Map complex paths (e.g., /mnt/disk_01/data) to simple ones (e.g., /data).
- **Flexibility:** Unlike Hard Links, Symlinks can point to **directories** and across **different disks**.

"user/symlinktest": User space test program to test the implementation of "symlink"

This program is a "**stress test**" for the new symlink system call and the modifications you made to open. It ensures that your kernel doesn't just create links, but actually traverses them correctly.

1. **The Core Objective**

The test checks if the kernel can handle three main scenarios:

- **Direct Links:** Does open("*link_to_a*") successfully open file "*a*"?
- **Chained Links:** If 3 -> 2 -> 1 -> file, does the kernel have the "patience" to follow the whole chain?
- **Cyclic Links:** If A -> B -> A, does the kernel detect the infinite loop and stop, or does it crash/hang?

2. **The Four Major Test Phases**

Test Phase	What it does	Goal

Basic Link	Createsa normal file "**a**", then a link file "**b**" pointing to "**a**".	To see if opening "**b**" gives the data inside "**a**".
Concurrent Links	Starts multiple processes creating and opening links at the same time.	To check if **locking logic** (*ilock/iunlock*) is correct and prevents "Race Conditions."
The Chain Test	Creates a long chain of links (Link 4 points to 3, 3 to 2, 2 to 1, 1 to file).	Tests whether while loop in "*sys_open*" can handle multiple steps of redirection.
The Circle Test	Creates Link A pointing to B, and Link B pointing to A.	Verifies the depth counter (usually limited to 10) prevents an **infinite loop**.

Implementation

Task requirement: This task involves two sub-tasks.

- Sub-task1: Implement "*symlink*" system call. This system call creates the "sticky note" on disk saying "go look at target. Follow Step 1 and Step 2 below.
- Sub-task2: Modify the "open" system call to read the path to reach the target file. Follow Step 3.

Step 1: Define the Type

- Define a new file type "T_SYMLINK" in "*kernel/stat.h*"
- Define "O_NOFOLLOW" flag in "kernel/fcntl.h". This flag tells "*open()*" to open the "*symlink*" itself rather than following the shortcut.

  Note: The flags passed to open are combined using a bitwise OR operator, so your new flag should not overlap with any existing flags.

- Register the System Call:

  Add "*SYS_symlink*" to "*kernel/syscall.h*"

  Add "*int symlink(const char*, const char*);*" to "*user/user.h*"

  Add "*entry("symlink");*" to "*user/usys.pl*"

  Add "*extern uint64 sys_symlink(void);*" to "*kernel/syscall.c*"

  Add *"[SYS_symlink] sys_symlink,"* to "*static uint64 (*syscalls[])(void)*" in "*kernel/syscall.c*"

Step 2: Create the "*symlink*" System Call in "*kernel/sysfile.c*"

Implement a new system call: "uint *symlink(void)*".

Use "*argstr*" to to get the two arguments (targer and path) passed to "*symlink(const char*, const char**)" from user space program *("user/symlinktest.c")*

target: The existing file that needs to be pointed to.

path: The name of the new shortcut file you are creating.

Task:

1. Fetch the target and path strings from the user.
2. Start a transaction (begin_op).
3. This system call should create a new "*inode*" with the "type = T_SYMLINK".

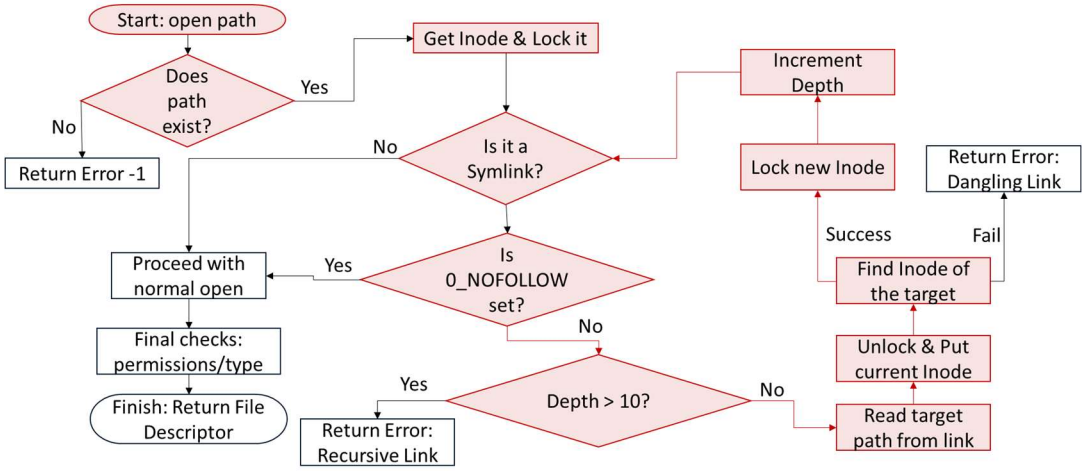
   create(path, T_SYMLINK, 0, 0)

4. Use "*writei()*" to save the target path string (e.g., "/usr/bin/sh") into the data blocks of this new "*inode*".
5. The "*inode*" itself doesn't need much else, it just "holds" that string for later.

Step 3: Update "*open()*" in "*kernel/sysfile.c*"

The most important part of this lab is modifying the "sys_*open()*" system call in "*kernel/sysfile.c*".

- When "*open()*" encounters a file of type "*T_SYMLINK*" and the "O_NOFOLLOW" flag is NOT set, it should not open the link itself.
  - Enter a loop to follow link.
  - **Read the Target**: Use "*readi()*" to read the path string in the "*symlink*" (the link's data blocks) and "jump" to the target file.
  - **Find New "inode"**: Call "*namei()*"on that new path.
    - If a "*symlink*" points to a file that was deleted, "*open()*" would return -1, because the target "*inode*" no longer exists.
    - **Recursive Links:** What if a link points to another link? Start over and find the "*inode*" for that path. The code must follow the chain until it hits a real file.
  - **Recursion Guard (Infinite Loops):** To prevent the kernel from spinning forever (e.g., Link A points to Link B, and Link B points to Link A), cap the depth (e.g., maximum 10 jumps).



Step 4: Update "*Makefile*"

Make sure "$U/_symlinktest\" is added to Makefile to compile.

Tips:

- To ensure the kernel functions correctly and follows the security logic of xv6, you should insert the symlink-following code after the initial "*ilock(ip)*" but before the directory check (*if(ip->type == T_DIR ...)*).

  After ilock(ip): You must have the lock on the "*inode*" before you can safely read its type or data blocks using "*readi()*".

  Before the Directory Check: Symbolic links can point to directories. If you perform the directory check before resolving the link, a "*symlink*" pointing to a directory might bypass or fail the check incorrectly.
- **Locking**: Be very careful with "*ilock()*" and "*iunlock()*". If you "follow" a link to a new "*inode*", make sure you release the lock on the current link before moving to the next one to avoid a deadlock.

Testing the task

The following example illustrates "*symlinktest*" behavior:

```
$ make clean && make qemu
xv6 kernel is booting

init: starting sh
$ symlinktest
Start: test symlinks
test symlinks: ok
Start: test concurrent symlinks
test concurrent symlinks: ok
```

Exit xv6 shell with keys "**Ctrl + a**" followed by "**x**".

Run all test cases

```
$ ./grade-lab-fs symlink
make: 'kernel/kernel' is up to date.
== Test running symlinktest == (2.3s)
== Test   symlinktest: symlinks ==
  symlinktest: symlinks: OK
== Test   symlinktest: concurrent symlinks ==
  symlinktest: concurrent symlinks: OK
$
```

Assignment submission

1. Execute "**make clean && make grade**" to test "*bigfile*" and "*symlink*" tasks.

```
$ make clean && make grade
== Test running bigfile ==
$ make qemu-gdb
running bigfile: OK (91.2s)
== Test running symlinktest ==
$ make qemu-gdb
(2.0s)
== Test   symlinktest: symlinks ==
  symlinktest: symlinks: OK
== Test   symlinktest: concurrent symlinks ==
  symlinktest: concurrent symlinks: OK
Score: 100/100
$
```

2. Execute "**make zipball**" to create "**lab.zip**".

3. Upload the "**lab.zip**" file to Gradescope assignment "**xv6labs-w8: fs**" to get auto graded.